PanelPilot — the assignment

Compile the medical rulebooks into a trustworthy engine, then productionize it into a system a clinic could pilot · 10 calendar days · all data open-source & synthetic

> **Read this first.** This assignment gives you a real problem, real source documents, a real user, and a quality bar — **deliberately NOT a feature list**. Deciding what to build, how deep, and in what order is the assignment. It will not all fit in the timebox; disciplined, documented cuts beat silent corner-cutting, and you will defend what you built *and* what you chose not to.

Part 0 · A 10-minute healthcare primer (no prior knowledge assumed)

The world you're building for. In the US, a primary-care clinic (a family doctor's practice) increasingly gets paid not just per visit but for *keeping patients healthy* — this is called **value-based care (VBC)**. Insurers score clinics on **quality measures**: simple, auditable questions like *"what % of your diabetic patients had their blood-sugar tested this year?"* Good scores mean bonus payments; bad scores mean lost revenue.

The core object: a care gap. One patient failing one measure — e.g., *"Maria, 62, diabetic, no HbA1c blood test in 14 months."* Think of it as a `lint` warning on a patient's record: specific, rule-generated, fixable by a concrete action (order the test, book the visit).

Your user: the care manager. A nurse/coordinator who works these gaps — calling patients, booking visits. A 5,000-patient panel produces thousands of gaps; they have ~2 hours a day for outreach. **Deciding what NOT to do today is the job.**

Glossary for the data: *encounter* = any visit · *observation* = a measurement (blood pressure, lab result) · *condition* = a diagnosis · *immunization* = a vaccine dose · *FHIR* = the standard JSON format health data ships in · *panel* = all patients attributed to the clinic. That's all the domain knowledge required; everything else is specified below. **Never use real patient data anywhere in this assignment.**

Part 1 · The situation

A family-practice clinic in Corpus Christi, Texas — ~10,000 patients, newly in value-based contracts — has two problems at once:

Problem A (the knowledge): the rules of preventive care live in long, unstructured authority documents (screening guidelines, immunization schedules with footnotes). The clinic follows them from memory. When a guideline changed recently (colorectal screening moved from age 50 to 45), thousands of patients newly qualified and nobody recomputed who. *"Make the rulebooks work for us — and we must be able to trust it; a wrong rule applied to 10,000 people is worse than no tool."*

Problem B (the operation): their EHR exports are messy — lab values without units, duplicated encounters, missing dates, and occasionally an entire data source silently stops arriving for weeks. One care manager has ~2 hours a day for outreach. They were burned by a dashboard tool before: 4,000 unexplained "gaps," ignored within a month, and nobody could tell whether outreach work ever resulted in care actually happening.

They ask you — a one-person product team — for **one working system they could realistically pilot:** the rulebooks go in; a trustworthy, prioritized, verified worklist comes out the other end.

Part 2 · The raw materials (all public)

Patients: ~10,000 via Synthea (github.com/syntheticchealth/synthea):

```
java -jar synthea-with-dependencies.jar -p 10000 Texas "Corpus Christi"
```

Synthea's output is clean; the clinic's reality above is not. How you make your evaluation honest about that is your decision — we will test your system with imperfect data at the walkthrough.

The rulebooks (all three in scope, including their ugly parts):

1. **USPSTF A & B recommendations** (uspreventiveservicestaskforce.org) — many are risk-based, not just age/sex.
2. **CDC child & adolescent immunization schedule** (PDF) — the footnotes carry real conditions.
3. **AAP Bright Futures periodicity schedule** (PDF grid).

A calibration set: the four simplified measures below are a fixed, unambiguous spec — implement them first as deterministic ground truth to prove your execution machinery before your compiled rules flow through the same pipeline. (These simplified versions ARE their spec; don't re-derive them.)

The four measures (your spec — simplified from public guidelines; these simplified versions ARE the spec)

ID	Who counts (denominator)	Passes if (numerator)
BP-CTRL	Adults 18–75 with a hypertension diagnosis dated before 2025-06-30	Most recent 2025 blood-pressure reading < 140/90 (both numbers)
A1C-TEST	Adults 18–75 with a diabetes diagnosis	≥1 HbA1c lab result in 2025 · **second rate:** most recent < 8.0%
CRC-SCREEN	Adults 46–75	Colonoscopy procedure 2016–2025 **OR** stool test (FOBT/FIT) in 2025
WELL-CHILD	Patients aged 3–21 all year	≥1 wellness / well-child encounter in 2025

Rules of the rules: measurement year is 2025 · age math at end of year unless stated · deceased-before-2025 excluded everywhere · **every verdict must be explainable** — the system must show which records made a patient pass or fail. At the walkthrough we pick 5 random patients and you prove the verdicts from raw data.

A free oracle: AHRQ's public ePSS / Prevention TaskForce API returns USPSTF recommendations for a given person-profile — useful for testing your compiled rules. Building your product as a thin wrapper around it answers none of the hard questions.

Part 3 · Your mission + the hard boundary

> Build the system YOU believe serves this clinic, end to end: documents compiled into rules → rules + calibration measures executed over the messy panel → a product the care team can trust and act on → operable enough to pilot. You own scope, architecture, stack, and interface. **Non-negotiables:** whatever runs against patients is deterministic and inspectable — same input, same output, **no model call deciding a patient's verdict at runtime** (AI compiles; rules execute) · a rule without a verbatim source quote is structurally impossible · every patient-level output traces to the records and the rule that caused it · setup in ≤3 commands · no real patient data, ever.

Part 4 · The hard questions (judged against these; how — and how deeply — is up to you)

On the knowledge side:

1. Your extractor produced a rule. How do you **know** it's real — and how do you *measure* how often extraction is right? ("It looked correct" is not evidence; decide what is.)
2. What makes a fabricated rule structurally impossible — not just unlikely — in your design?
3. The footnotes and risk-based criteria are where the real medicine lives. How much does your representation capture, and what did it genuinely fail to capture? (We will look.)
4. A guideline updates (50 → 45). What happens in your system? Who finds the patients that newly qualify?
5. Where is AI genuinely load-bearing in your system, and where did you deliberately refuse it — argued from failures you actually observed?

On the operation side:

6. A lab value arrives without units; a visit is duplicated. What does your system do — and what does it *never* do?
7. The lab feed silently dies for three weeks. What does the clinic see? (The worst answer is "nothing changes.")
8. New data arrives nightly. What recomputes, and what does that cost at 10,000 patients — and at 100,000?
9. The care manager marks a patient "handled." How does the clinic ever know the care **actually happened**? What if it didn't?
10. Can she open your product and know how to spend her 2 hours — in under 2 minutes? With 20 contacts/day across 10,000 patients, who's first, and can you say *why them* in one sentence each?
11. Another team builds a mobile app on your system next quarter. What do you hand them?
12. Six months in, what numbers prove to the clinic that your product is working?

Twelve questions will not all get deep answers in the timebox — that is intentional. Choosing which to go deep on, and saying why, is the test.

Part 5 · What you submit

- **The working system** — repo, README (setup ≤3 commands · architecture in ≤1 page · your rule representation and why · your evidence of extraction quality, however you chose to establish it · what recomputes nightly and its measured cost)
- **A ≤10-minute demo video** — documents in → trust established → messy panel executed → the care manager's worklist → one full verify-or-reopen cycle
- **The decision memo (≤2 pages — weighted like code):** what you built and why · what you consciously did NOT build and why · which hard questions you went deep on vs punted, honestly · what you'd build next with another two weeks
- Your AI-usage log

Timebox: 10 calendar days.

AI tool policy — this is signal, not a loophole

You are **encouraged** to use AI assistants heavily, with your own tooling. For the document-processing pipeline we can provide an Anthropic API key with a **$25 hard spend cap** on request — the corpus is small, and $25 is genuinely sufficient if you work sensibly (prompt caching, per-section extraction, batching); treat the budget as part of the engineering problem and **report your total token spend in your AI-usage log**. Conditions: (1) the AI-usage log — tools, what for, 2–3 prompts that mattered, one place the AI was wrong and you caught it; (2) a **60-minute live walkthrough** where you defend any line or decision — "the AI wrote it" is an acceptable origin and an unacceptable explanation; (3) the decision memo must be yours.

How you'll be evaluated (shared openly)

Dimension	Weight
Product translation — did you choose the right things to build for this user, and can you defend every inclusion AND exclusion?	25%
Correctness with evidence — rules provable from the documents, verdicts provable from the records, when we spot-check both	25%
The two cores — extraction trustworthiness (gold evidence, hallucination defense) AND production behavior (bad data, dead feeds, recompute cost, verification)	25%
Judgment under the hard questions — how many addressed, how honestly	15%
Communication — README, demo, memo	10%

Honesty multiplier: a stated limitation costs nothing; a hidden one we discover costs double. There is no feature checklist — there is a user to serve and questions to survive. Build something a nurse would thank you for.